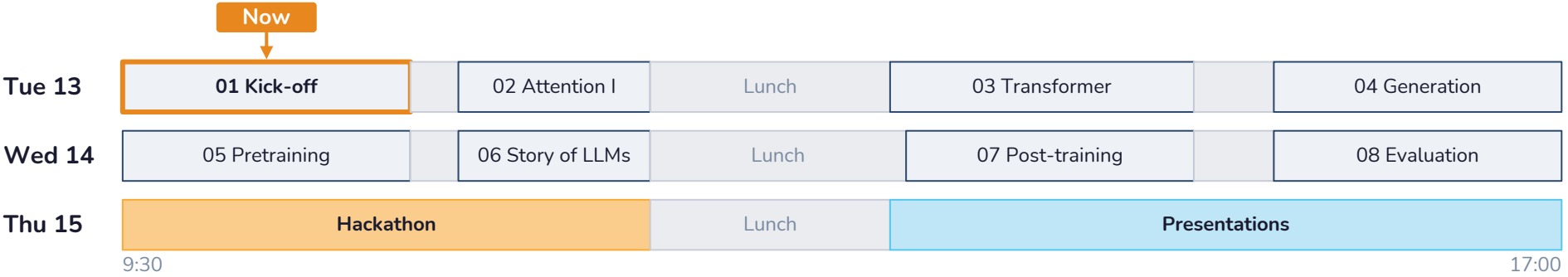
Advanced NLP

Session 01 · Kick-Off & Text Data

Prof. Dr. Richard Sieg · TH Köln IWS · WS 26 / 27
Tuesday, October 13, 2026

Three days, one model

Day	Morning	Afternoon
Tue 13	Kick-off & text data · Attention I	Attention II & the Transformer block · Weights & generation
Wed 14	Pretraining · The story of LLMs	Post-training · Evaluating LLMs, project kick-off
Thu 15	Hackathon on your component	Presentations



9:30 to 17:00 each day. Breaks at 11:00, 12:15 for lunch, and 15:05.

By Thursday evening, you can



build a GPT-2 from scratch and load OpenAI's real weights into your own code



pretrain it and read what its loss curve tells you



align it with supervised fine-tuning and preference tuning



change one component of a modern LLM and measure what it does



judge whether a measured difference is an effect or noise

Agenda

01. Organization

02. Where language models come from

03. What a language model sees

04. From tokens to training data

05. Appendix

01.

Organization



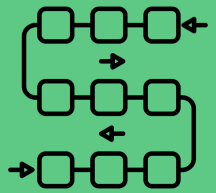
About Me

- Professor for NLP & LLMs since Dec 2025
- Since 2023 external lecturer for NLP @ IWS
- Mathematics Background
- NLP & ML Engineer for eight years
- Many years of university level teaching
- richardsieg.github.io

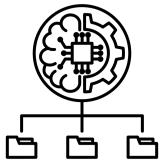
We build GPT-2, train it, align it, and then take it apart

- ***Tuesday*** you write a GPT-2 from scratch, one class at a time, and finish by loading OpenAI's original weights into your own code.
- ***Wednesday*** you pretrain it on children's stories, teach it to follow instructions, and tune it towards preferred answers.
- ***Thursday*** you swap one component for the version modern LLMs use, and measure what changes.

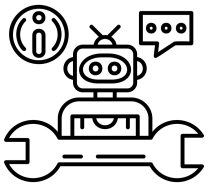
Three ingredients make an LLM



Process long sequences and context



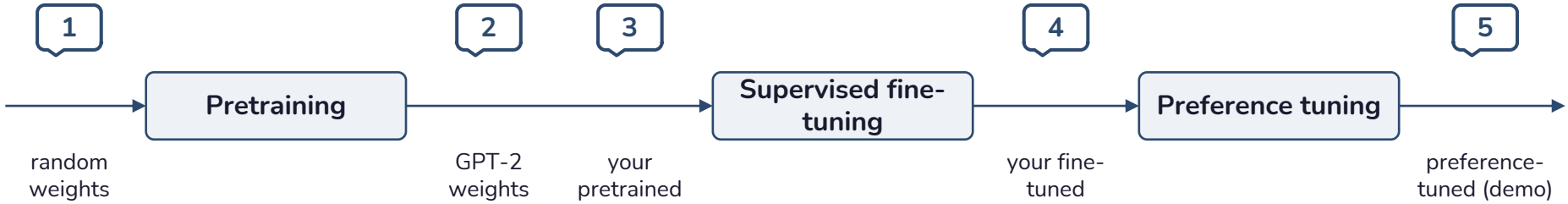
Efficient training on huge datasets



Follow (human) instructions

Five times this week, you talk to a model

- On day 1 you build a small chat window in marimo. It never changes after that.
- What changes is the model behind it, five times: random weights, OpenAI's GPT-2 weights, your own pretrained model, your own fine-tuned model, and a preference-tuned model in a demo.
- The middle two will ignore your question and just keep writing. That is the point.



The same chat window, five different models. You will see the difference between pretraining and fine-tuning before you see it in a formula.

Register for the exam in PSSO **TODAY**!

Your grade

- ***Presentation, Thursday afternoon, 60 percent.*** 12 minutes per pair, then 6 minutes of questions. Questions go to one partner at a time, and both of you will be asked.
- ***Technical report and repository, due October 21, 23:59, 40 percent.*** At most four pages, plus an appendix on how you used AI tools.
- Eight criteria, each scored 0 to 6. Four are scored for you alone, which is a third of your grade.
- The full rubric and description is in the repository.

What gets graded is the experiment, not its result

- On Wednesday evening every pair gets one component that modern LLMs changed relative to GPT-2
- On Thursday you implement it in your own model and compare it against the unmodified baseline, each at three random seeds.
- Whether your component helps is not graded. Whether you can tell is.

RMSNorm	SwiGLU
Sliding-window attention	QK-Norm
Sinusoidal positions	RoPE
Grouped-query attention	KV cache

Use AI tools, and expect to explain every line

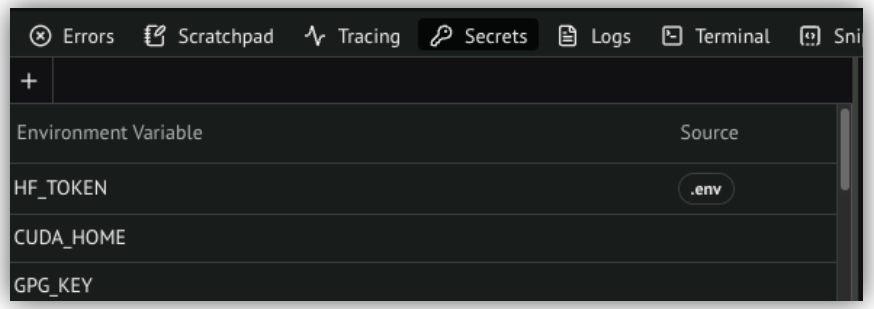
- Use whatever helps: Claude Code, Copilot, ChatGPT.
- Your report has an appendix saying which tool you used, for what, and what you changed afterwards. Writing it costs no points. Leaving it out does.
- On Thursday the questions come at your own machine: “show me the line where this happens”, “what if you remove it”.

You work in pairs

- Pairs for all three days, from now until Thursday's presentation. Maybe one trio.
- Each of you has your own GPU session, so a long run never blocks your partner.
- During the coding sessions, let’s create an “AI free environment”. I will support all pairs.
- Solutions to the notebooks are already public.
- At the end of each block, you can use the solution from our repo, so nobody starts the next block from broken code.
- On Thursday both of you present and both of you are asked. Work so that either of you can explain any part.

Your setup: one repository, one molab session, one token

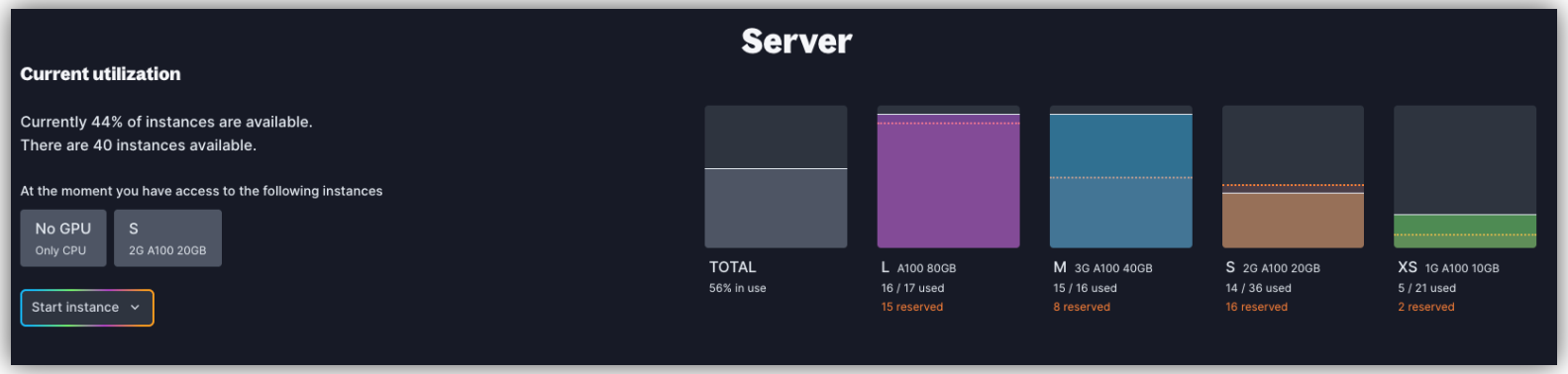
- ***Your team repository*** in `th-koeln-iws-nlp`, one per pair.
- ***molab*** is where the GPU work runs. Your course access is set up before Tuesday. Your Hugging Face token lives in its Secrets panel, never in a notebook cell.
- ***Backup:*** the GPU cluster in Mainz, through JupyterHub, with persistent storage (see next slide).
- ***Saving your work:*** at every break, download the notebook from molab, commit and push.



molab storage: marimo.io/pages/molab/storage

Other Ways for GPU

1. Our institute has access to an HPC in Mainz and I can give you access to it. I already reserved GPU for most of you.
2. Google Colab in the free version has an okay-ish GPU and for 10$ per month you can get decent ones.
3. You have a PC with a GPU and/or a MacBook Pro with Silicon Valley Chip.



marimo

- An alternative to Jupyter that lives in plain Python files, so diffs and imports work.
- ***Reactive:*** change a cell, and every cell that depends on it runs again, automatically.
- So every variable is defined in exactly one cell. Defining it a second time is an error, not a silent overwrite.
- Lots of possibilities for inputs and outputs
- Start the editor with `uv run marimo edit`. Our notebooks also import classes from each other, which is why every class sits alone in its own cell.
- I am a marimo ambassador now 🎉

Richard Sieg

TH Köln · Machine Learning & NLP

Teaches machine learning and NLP at TH Köln in Cologne. Uses marimo for interactive concept playgrounds, student assignments with automated code review, and course slides.



Pair up and check your setup

10 minutes

1. Find your partner.
2. Enter the details in the form.
3. Accept the invitation to your team repository and clone it.
4. Both of you: open notebook 00 on molab and confirm it shows a GPU.



02.

Where language models come from

You already know the classic NLP toolbox

- Text processing (spaCy, RegEx)
- Relations and semantics
- Frequency-based embeddings (BoW, tf-idf)
- Prediction-based word embeddings (word2vec)
- Information extraction
- Text classification
- Language modeling

Discriminative vs Generative Tasks

Discriminative tasks

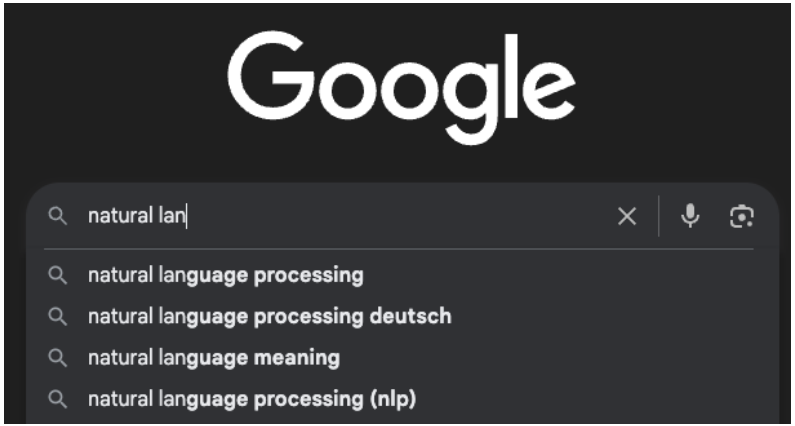
- Text classification (e.g. sentiment)
- Named entity recognition
- Part-of-speech tagging
- Semantic role labeling

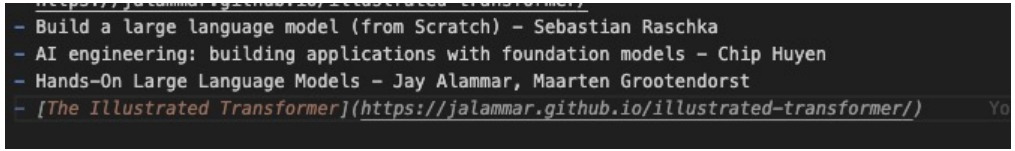
Generative tasks

- Chatbots
- Text completion
- Machine translation
- Question answering
- Summarization
- Simplification

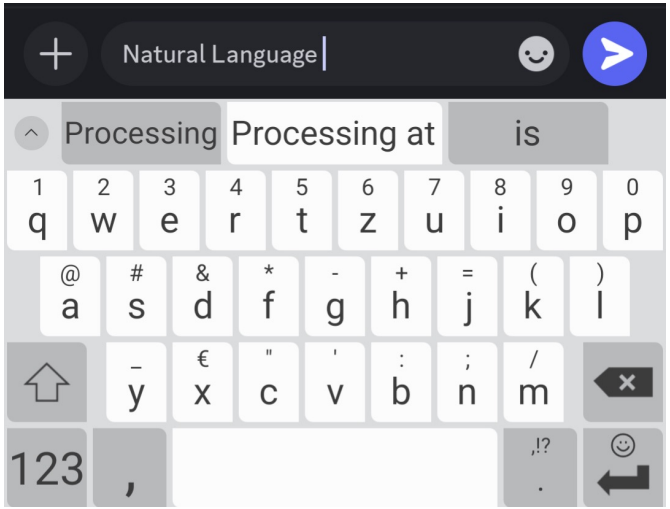
A language model answers one question, over and over: what comes next?

Language modeling is everywhere









Language modeling is next-token prediction

- A ***language model*** is a statistical model that assigns a probability to a sequence of words, more precisely of ***tokens***:

$$p(w_1, \dots, w_m)$$

- Related task: the probability of the next token given the previous ones

$$p(w_t | w_{t-1}, \dots, w_{t-n-1})$$

- ***Chain rule***

$$p(w_1, \dots, w_m) = \prod_{i=1}^{m} p(w_i | w_1, \dots, w_{i-1})$$

- p(be water my friend) = p(be) · p(water | be) · p(my | be water) · p(friend | be water my)

An n-gram model only looks at the last few tokens

- We can approximate the conditional probability by only looking at the past n tokens (***n-gram model***)
  - p(friend | be water my) ≈ p(friend | my) (*bigram*)
- ***Bigram model:*** predict the next word by only looking at the previous word:

$$p(w_i|w_1, \dots, w_{i-1}) \approx p(w_i|w_{i-1})$$

- Calculation is based on counting frequencies:

$$p(w_t|w_{t-1}, \dots, w_{t-n-1}) = \frac{count(w_{t-n+1}, \dots, w_{t-1}, w_t)}{count(w_{t-n+1}, \dots, w_{t-1})}$$

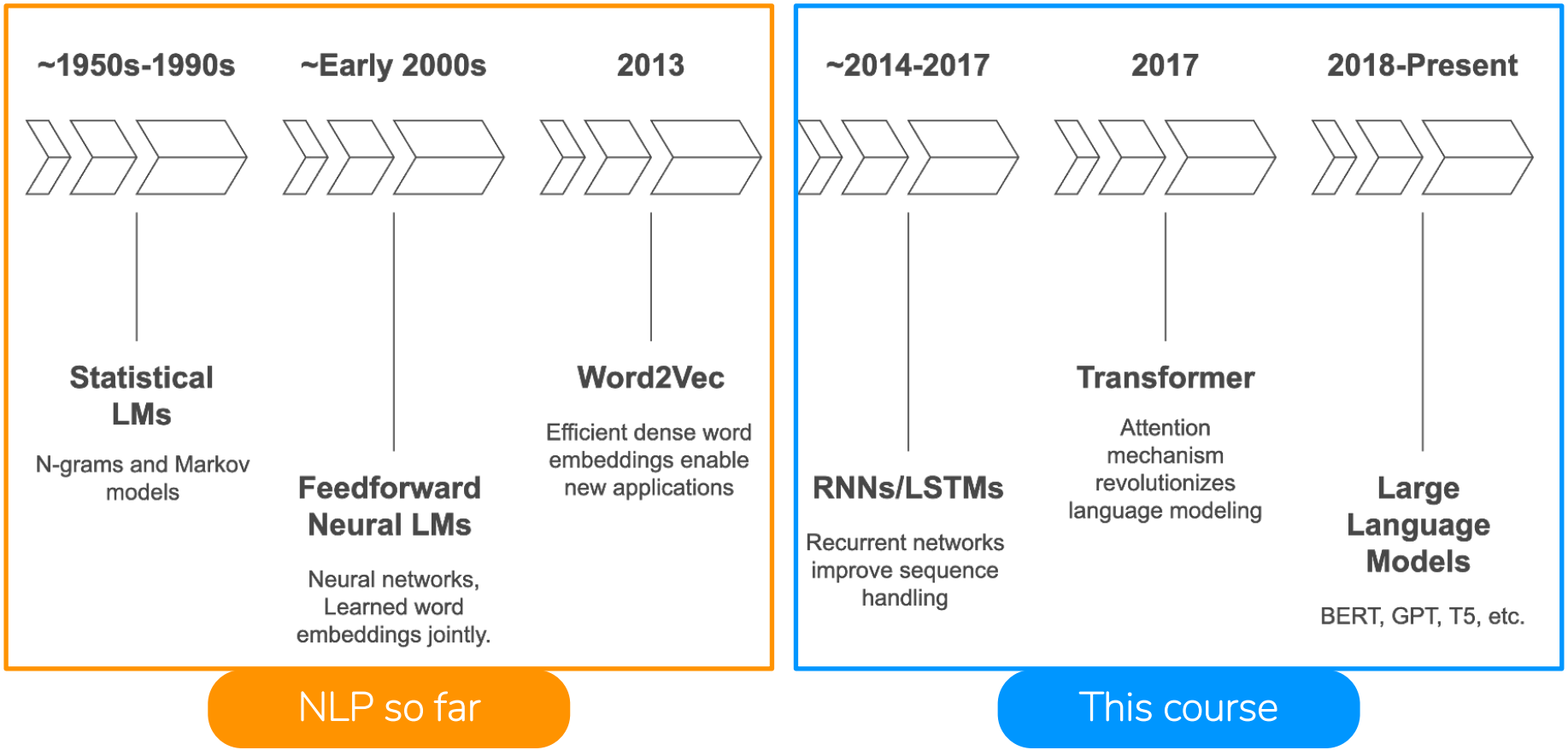
Counting n-grams breaks

- The Markov assumption can be used to build a language model by computing n-gram probabilities over a large corpus

$$p(w_t|w_{t-1}, \dots, w_{t-n-1}) = \frac{count(w_{t-n+1}, \dots, w_{t-1}, w_t)}{count(w_{t-n+1}, \dots, w_{t-1})}$$

- ***Issue 1: Sparsity.*** If a sequence of words rarely or never occurs in the corpus, its probability is almost 0
- ***Issue 2: Storage.*** We need to store all occurrences of all sequences of tokens
- Increasing the window n makes both issues worse. More than 5 is impractical.

History of Language Models



You shall know a word by the company it keeps.

J.R. Firth, 1957

Frequency-based embeddings count words

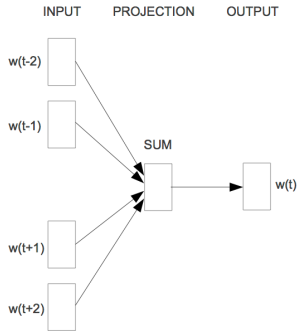
- Given: a number of documents
- ***Bag of Words:*** count how many times a word appears in a document
  - vector representation of one document
  - vector dimension: size of the vocabulary (unique words)
- ***Term Frequency - Inverse Document Frequency***

$$\mathrm{tf}(t,d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}} \qquad \mathrm{idf}(t,D) = \log \frac{|D|}{|\{d \in D : t \in d\}|}$$

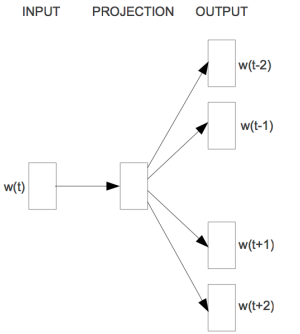
$$\mathrm{tfidf}(t,d,D) = \mathrm{tf}(t,d) \cdot \mathrm{idf}(t,D)$$

word2vec learns fixed vectors by predicting its neighbors

- Learn a vector representation of a word by predicting…
  - the word itself based on its context (***Continuous Bag of Words***)
  - the context words based on a target word (***Skip-Gram***)
- Word2vec, fastText, GloVe
- Issue: generates fixed embeddings, the neural network itself cannot be used for further task-specific fine-tuning



CBOW



Skip-Gram

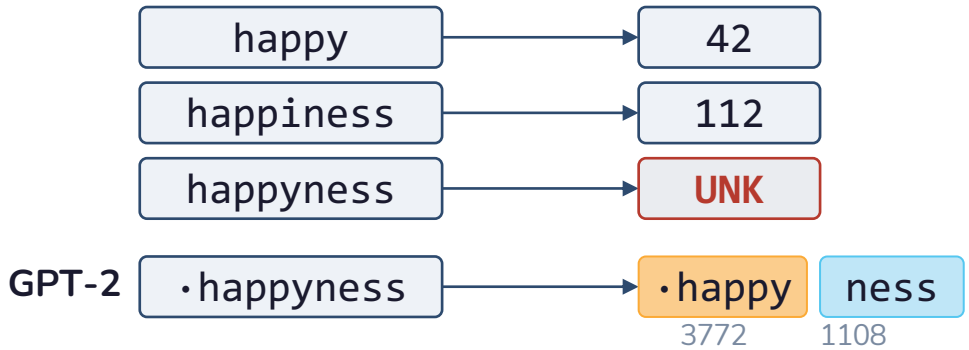
Mikolov et al. 2013: arxiv.org/abs/1301.3781

03.

What a language model sees

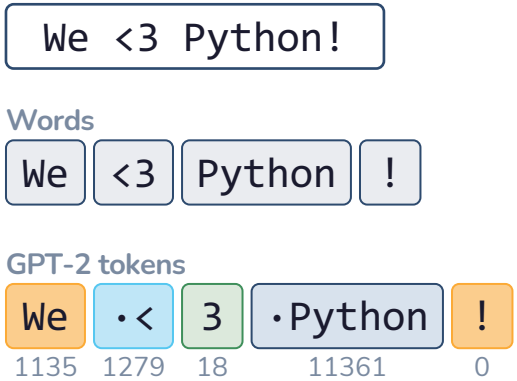
Fixed Vocabulary

- So far, we gave every known word an index. Unknown words became ***UNK***.
- ***Issue 1:*** the vocabulary grows huge.
- ***Issue 2:*** it cannot use what words share, “happiness” has nothing to do with “happy”.
- With ***subword tokens***, the misspelling is no longer unknown. It is two pieces you already know.



Tokens are usually pieces of words

- ***Token:*** a sequence of characters, mostly a whole word or a part of one.
- ***Tokenization:*** splitting raw text into tokens.
- `We <3 Python!` as words: `We` · `<3` · `Python` · `!`
- As GPT-2 tokens: `We` · `·<` · `3` · `·Python` · `!`, with · marking the space that belongs to the token

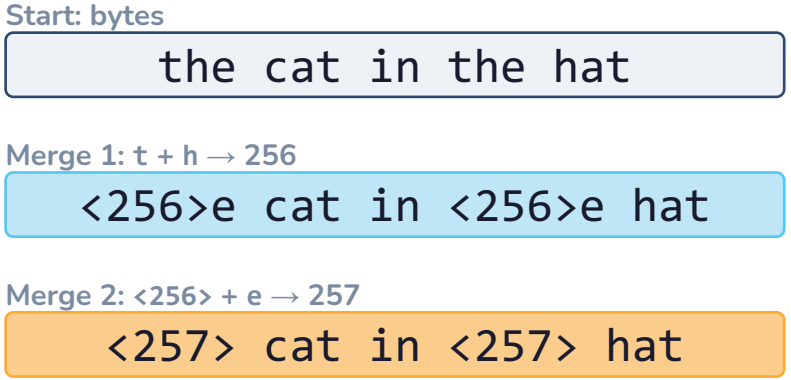


" happy" and "happy" are two different tokens with two different IDs. The space is part of the word.

Tiktokenizer: tiktokenizer.vercel.app

BPE merges the most frequent pair, again and again

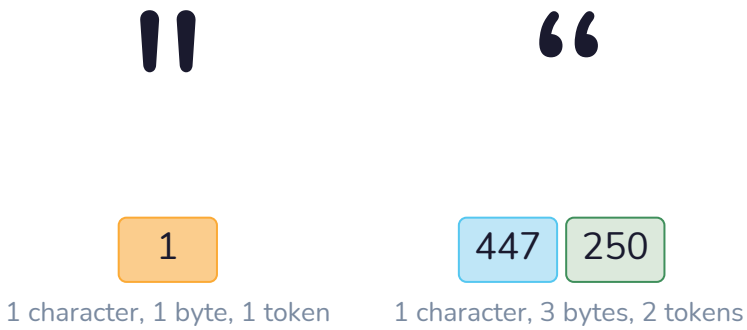
- ***Goal:*** a subword vocabulary of a fixed size. GPT-2's has 50,257 entries.
- Start with the 256 possible byte values. Every text is already a sequence of these.
- Count every pair of neighbors and merge the most frequent one into a new token. Replace it everywhere.
- Repeat until the vocabulary is full. The list of merges, in order, is the tokenizer.



Raschka, BPE from scratch: sebastianraschka.com/blog/2025/bpe-from-scratch.html

GPT-2 starts from bytes, so nothing is ever unknown

- UTF-8 stores every character as one to four bytes, and all 256 byte values are in GPT-2's vocabulary.
- So any text can be encoded. There is no unknown token.
- Common sequences become single tokens, rare ones stay split: `someunknownPlace` becomes `·some` · `unknown` · `Place`.
- The straight quote " is one token. The curly quote “ is one character, three bytes, and ***two*** tokens.



Learning Tokenizers

Wóŕð	ĞÞT–2	ĞÞT–4ó
Ħóćħśćħúĺƀıƀĺıótħéǩ	8!tóǩéńś	6 tokens
Śþŕáćħṃóðéĺĺ	4!tóǩéńś	2 tokens
Dóńáúðáṃþgśćħıgggáħŕtśğéśéĺĺśćħágŧ	14!tóǩéńś	9 tokens

- Each tokenizer learned its merges from its own training data. GPT-2's merges barely cover German.
- More tokens per word means less text fits into the same context, and every token costs compute.

GPT-2 Spr | ach | mod | ell

GPT-4o Sprach | modell

tiktoken gives us GPT-2's tokenizer in three lines

```
import tiktoken

tokenizer = tiktoken.get_encoding("gpt2")
ids = tokenizer.encode(
    "Once upon a time, there was a little girl named Lily.")
# [7454, 2402, 257, 640, 11, 612, 373,
#  257, 1310, 2576, 3706, 20037, 13]
tokenizer.decode(ids)
```

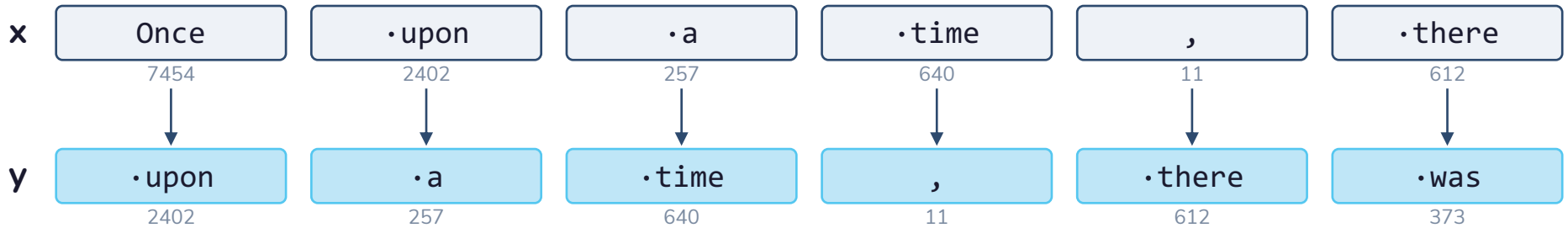
- encode turns text into IDs, decode turns IDs back into text.
- Vocabulary of 50,257 tokens. <|endoftext|> is the last one, 50256, and it separates the stories in our data.

04.

From tokens to training data

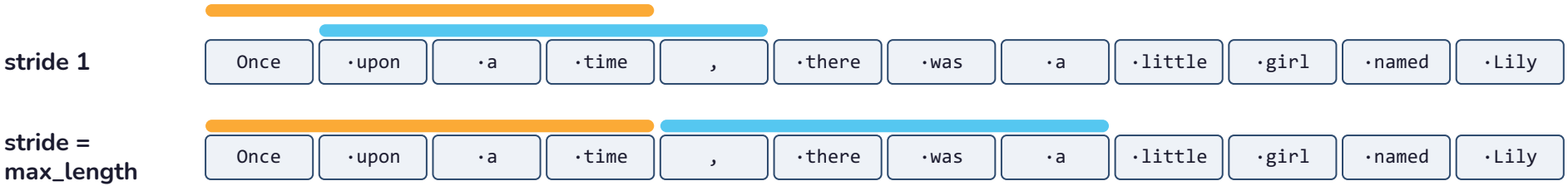
The target is the input, shifted by one token

- A training example is a window of tokens, ***x***, and the same window moved one token to the right, ***y***.
- At every position, the model sees x up to there and has to predict the token in y below it.
- A window of 512 tokens is 512 prediction tasks at once.



Sliding Windows

- The data loader in Raschka's book slides a window of `max_length` tokens over the text.
- `stride` is how far it moves each time. Stride 1 overlaps almost completely; stride equal to `max_length` does not overlap at all.
- More overlap means the model sees the same tokens more often, which can make it overfit.



We draw windows at random from 541 million tokens

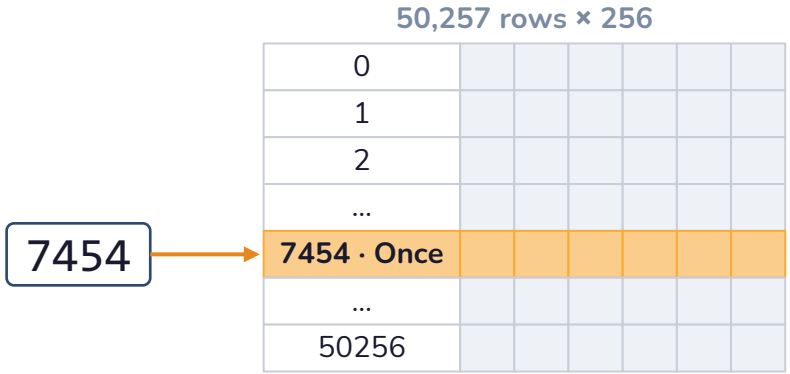
- Our training set has 541M tokens. Listing every window would be pointless.
- `get_batch` picks 64 random start positions. From each it cuts 512 tokens for x, and the 512 starting one token later for y.
- The file stays on disk. `np.memmap` reads only the windows we ask for.
- Same windows as on the last slide, with the positions drawn at random.



A language model answers one question, over and over: what comes next?

A token ID is an index into a table the model learns

- The model never treats 7454 as a number. It is a row number.
- Its first layer is a table with one learned vector per token: 50,257 rows, 256 numbers each in our model.
- In block 2, attention works on these vectors.



Notebook 01: from text to training windows

35 minutes, in pairs

1. Open notebook 01 on molab.
2. Encode and decode a TinyStories sample. Find a word that needs more than one token.
3. Implement `get_batch`. The shape check below the cell turns green when the shapes are right.
4. Decode one x and its y, and check by eye that y is x moved by one token.
5. Commit and push before 11:00.

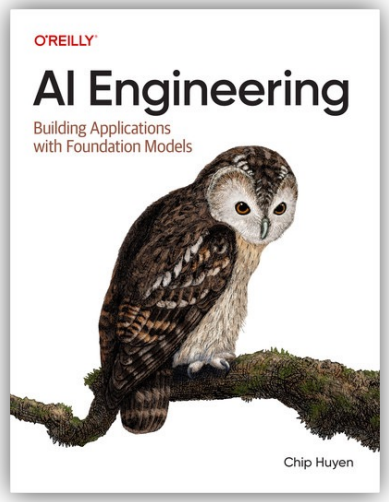
- ☐ `get_batch` passes its shape check
- ☐ You have seen one x and its y decoded side by side
- ☐ Your work is pushed to the team repository

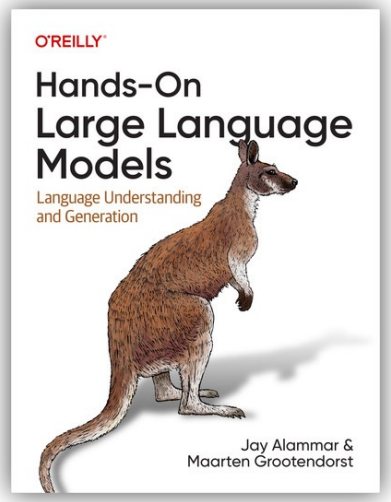
05.

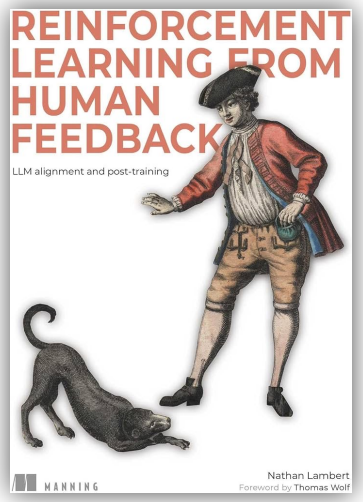
Appendix

Some literature









All textbooks are available online through the TH Köln library.

Folks to follow

- Godfathers of AI: Geoffrey Hinton, Yoshua Bengio, Yann LeCun
- Andrew Ng: Coursera founder, newsletter The Batch
- Andrej Karpathy: great videos on building LLMs, including the tokenizer
- Chip Huyen
- Jay Alammar
- Sebastian Raschka: his newsletter Ahead of AI is the best ongoing source for this course
- Vincent Warmerdam: talks and videos, now at marimo
- Ines Montani: founder of spaCy
- More newsletters and blogs: `richardsieg.github.io`

Sources

Literature

- Raschka, Build a Large Language Model (From Scratch), chapter 2: tokenization, BPE with tiktoken, the vocabulary of 50,257 and ID 50256, input and target pairs, the stride data loader. Used on the token, tiktoken, target and window slides.
- Alammar and Grootendorst, Hands-On Large Language Models, chapter 2: word, subword, character and byte tokens, byte fallback, the three factors that decide a tokenizer. Chapter 12: the three training steps. Used on the tokenizer and pipeline slides.
- Sennrich, Haddow, Birch 2016, Neural Machine Translation of Rare Words with Subword Units, arxiv.org/abs/1508.07909. Word-level BPE.
- Radford et al. 2019, Language Models are Unsupervised Multitask Learners. Byte-level BPE.

Literature, continued

- Jurafsky and Martin, Speech and Language Processing, 3rd edition draft, chapter 3: n-gram language models. Background for the n-gram slides.
- Mikolov et al. 2013, Efficient Estimation of Word Representations in Vector Space, arxiv.org/abs/1301.3781. The word2vec slide and its two diagrams.
- Firth 1957, the quote on the distributional hypothesis.

Code and courses

- tiktoken, github.com/openai/tiktoken
- Karpathy, Let's build the GPT Tokenizer, youtube.com/watch?v=zduSFxRajkE
- Raschka, Byte Pair Encoding from scratch, sebastianraschka.com/blog/2025/bpe-from-scratch.html. The two merge steps on the BPE slide.
- Tiktokenizer, tiktokenizer.vercel.app

Not presented, included in the handout.

Sources, checked numbers

Checked numbers, as of Oct 2, 2026

- Every token split and ID on these slides: GPT-2 vocabulary `r50k_base`, identical to tiktoken's `gpt2`, run through the `gpt-tokenizer` package. Cross-checked against Raschka's own example, which reproduces exactly.
- GPT-4 and GPT-4o counts: `cl100k_base` and `o200k_base`, same package.
- 3.95 characters per token, 1.29 tokens per word: one story from our training set, 727 characters, 184 tokens. An example, not a property of the dataset.
- 541,469,395 training tokens: our dataset on the Hugging Face Hub.

Documentation

- molab, storage and secrets, marimo.io/pages/molab/storage

Not used

- “About three characters per token” (Alammar): a general remark; our own data measures close to four.
- Vocabulary sizes of the GPT-4 and GPT-4o tokenizers: not checked.
- BPE “starting from all ASCII characters with an end-of-word symbol”: correct for word-level BPE, wrong for GPT-2.

Not presented, included in the handout.